



Movie Review Sentiment Analysis using NLP

From raw text to meaningful sentiment insights



The Dataset

Our sentiment analysis model was built using a robust and widely-recognised collection of movie reviews.



Overview

- IMDB Movie Reviews Dataset
- 50,000 reviews in total
- Balanced: 50% Positive / 50% Negative



Details

- Source: Kaggle public dataset
- Task: Binary classification (positive/negative)
- Data Type: Unstructured text



Why This Dataset?

- Large & reliable volume of data
- Balanced distribution minimises bias
- Standard benchmark in Natural Language Processing (NLP)

Problem & Key Challenges

Classifying movie reviews into positive or negative sentiment is a core task in NLP, but the unstructured and often nuanced nature of text data presents significant hurdles.



HTML Noise

Removing extraneous HTML tags, URLs, and special symbols is crucial for clean data input.



Sarcasm & Negation

Interpreting phrases like "not good" or "great... not!" accurately requires sophisticated linguistic understanding.

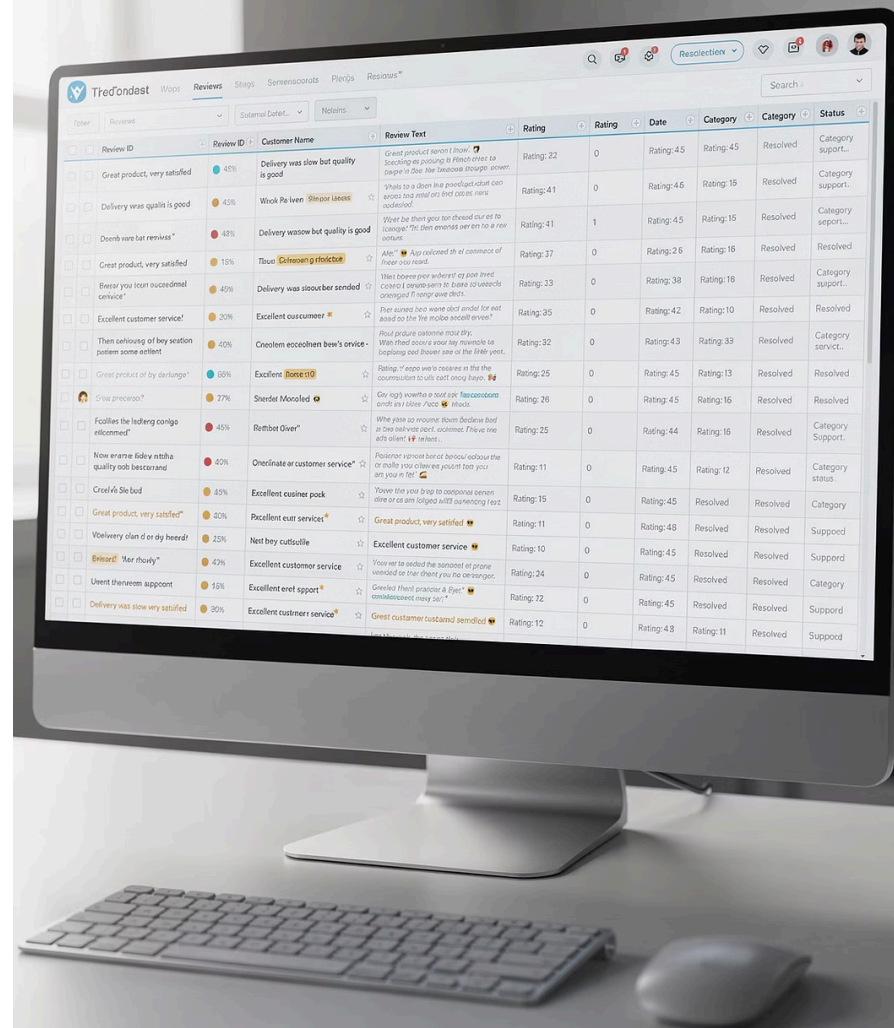


Mixed Sentiment

Reviews with both positive and negative aspects, e.g., "good acting but bad plot," pose a challenge for binary classification.



Text meaning is not always explicit — language complexity is the core difficulty.



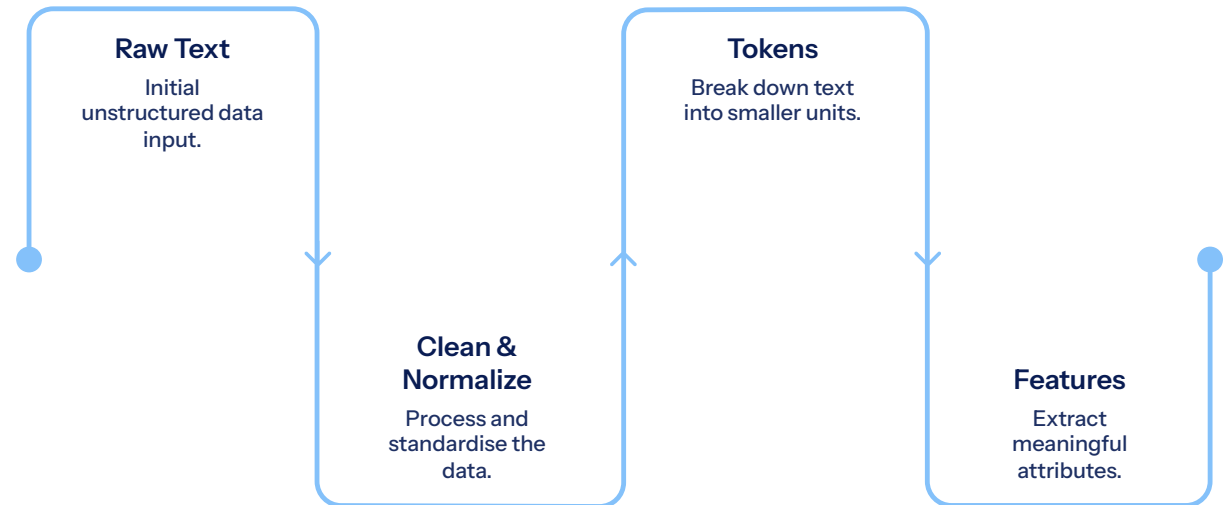
Preprocessing and Pipeline

Transforming raw, noisy movie reviews into a clean, structured format ready for machine learning models is a critical initial phase.

Preprocessing Steps

- Remove **HTML & URLs**: Eliminating web-specific tags and links that don't contribute to sentiment.
- Remove **Special Characters**: Stripping punctuation and symbols to focus on textual content.
- Convert to **Lowercase**: Standardising case to treat words uniformly (e.g., "Great" and "great" are the same).
- **Tokenization & Stopword Removal**: Breaking text into individual words and discarding common, low-information words like "the," "is," "a."
- **Lemmatization**: Reducing words to their base/dictionary form to standardise vocabulary.

Data Processing Pipeline



Each step in the pipeline refines the data, progressively making it more suitable for feature extraction.

Before vs. After Preprocessing

Preprocessing removes noise and reduces text to only the meaningful content the model needs.

Before: "GREAT film!!!
 <http://imdb.com>"

After: "great film"

Before: This film was ABSOLUTELY WONDERFUL!!!
 The acting=10/10 <http://imdb.com>

After: film absolut wonder act

✓ Noise removed → meaningful text only. Clean input directly improves model accuracy.

Feature Engineering — TF-IDF & Bag of Words

Feature Strategies Compared

TF-IDF | 89.47%

- Importance-based weighting
- Highlights **meaningful words**
- Reduces common words impact
- Better performance**

Bag of Words | 86.85%

- Counts word frequency only
- No weighting
- Simpler but less accurate
- Lower performance (-2.62%)**

N-grams — Capturing Context

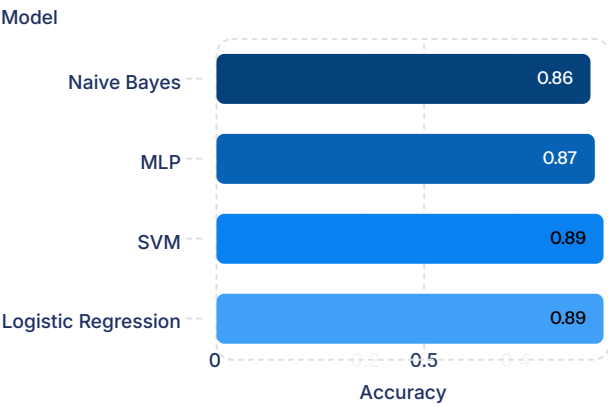
N-grams extend simple word counts to capture multi-word sequences, which is essential for understanding negation and contextual meaning.



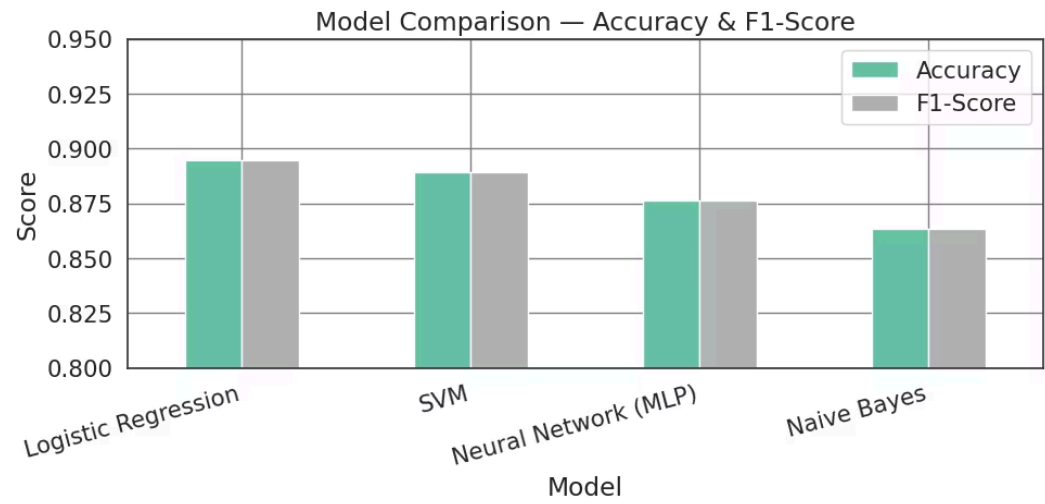
  N-grams capture context and improve negation understanding — a critical advantage over single-word models.

Models & Results

Four models were evaluated using TF-IDF features. Logistic Regression achieved the best performance.

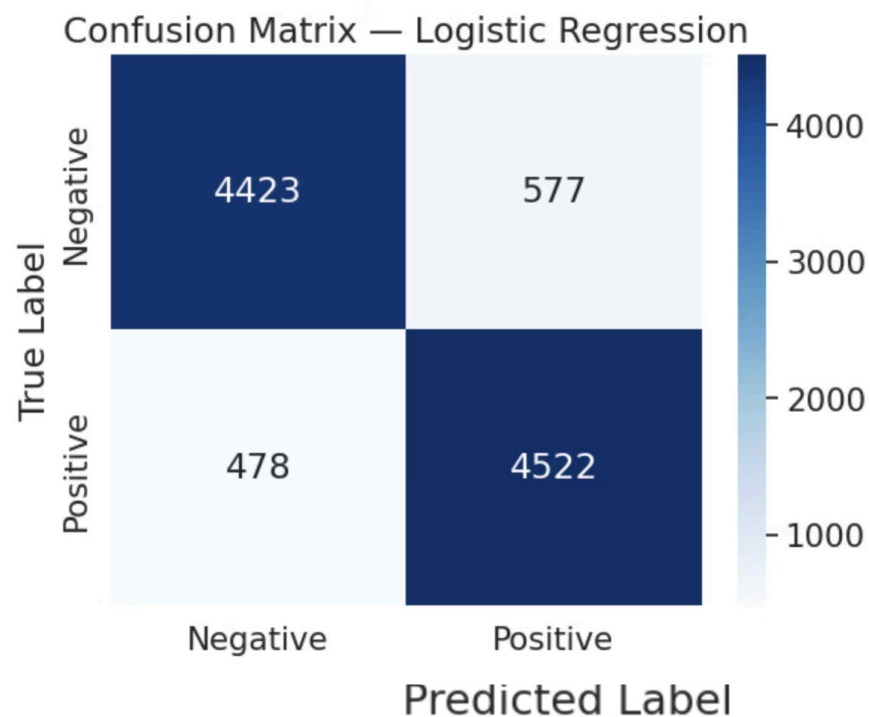


Model	Accuracy	F1-Score
Logistic Regression	89.46%	0.895
SVM	88.63%	0.886
Neural Network (MLP)	87.13%	0.871
Naive Bayes	86.37%	0.864



✔ Best model: Logistic Regression with 89.46% accuracy and F1-score of 0.895.

Evaluation



True Positives (correct positive): 4,522
True Negatives (correct negative): 4,423
False Positives (negative → positive): 577
False Negatives (positive → negative): 478

Confusion Matrix Insights

- ✓ High True Positives & True Negatives — the model correctly identifies the majority of both sentiment classes.
- ⓘ False Positives $\approx$ False Negatives — a well-balanced error distribution with no systematic bias towards either class.

Most Influential Features

Positive Signals

excellent, amazing, brilliant, outstanding, masterpiece

Negative Signals

bad, boring, terrible, waste, awful, disappointing

Error Analysis

Understanding where the model fails helps identify the boundaries of classical NLP approaches.



Short Reviews

Very brief reviews provide insufficient context for reliable classification — sparse feature vectors reduce confidence.



Mixed Sentiment

Reviews that praise some aspects while criticising others confuse the binary classifier — nuance is lost.



Negation

"Not good" and similar constructions are misclassified without deeper linguistic understanding of phrase-level context.



Sarcasm

Ironic praise like "Oh, just brilliant" carries hidden meaning the model cannot detect from surface features alone.



Errors are caused by language complexity, not model weakness.

Cross-Validation

Cross-validation confirms that the model's performance is stable and generalises well to unseen data.

89.33%

Mean CV Score

Average accuracy across all cross-validation folds.

89.45%

Test Accuracy

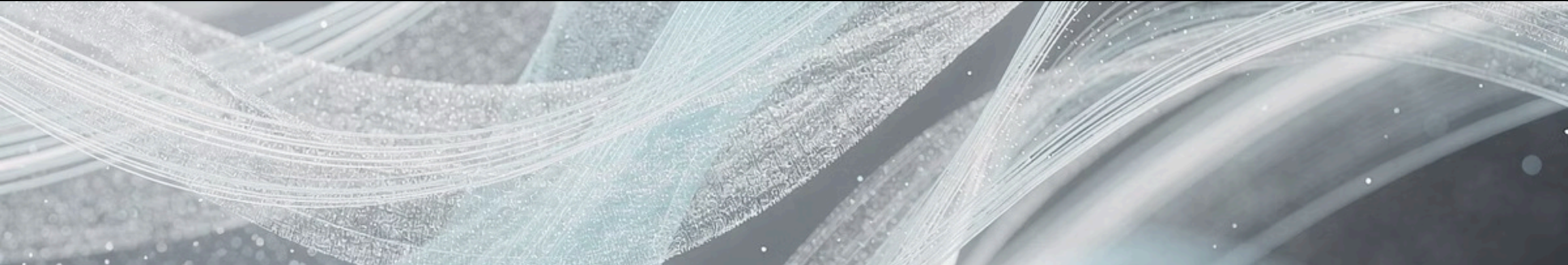
Final accuracy on the held-out test set.

0.12%

Difference

Negligible gap between CV and test performance.

👍 Model generalises well and shows no signs of overfitting — CV and test scores are virtually identical.



Key Findings

Best Model

Logistic Regression achieved the highest accuracy (~89%) and F1-score (0.895) across all evaluated models.

Best Features

TF-IDF combined with n-grams delivered the strongest feature representation, outperforming Bag-of-Words by +2.62%.

Stable Performance

Cross-validation mean of 89.33% vs. test accuracy of 89.45% — a gap of just 0.12%, confirming no overfitting.

Balanced Predictions

Error distribution is symmetric across positive and negative classes — no systematic bias toward either label.

Future Work

Classical ML approaches have proven effective, but the next frontier requires moving toward contextual and semantic understanding.



Multi-class Sentiment

Extend beyond binary to fine-grained ratings (1–5 stars) for richer insight.



Word Embeddings

Replace TF-IDF with Word2Vec or GloVe to capture semantic similarity between terms.

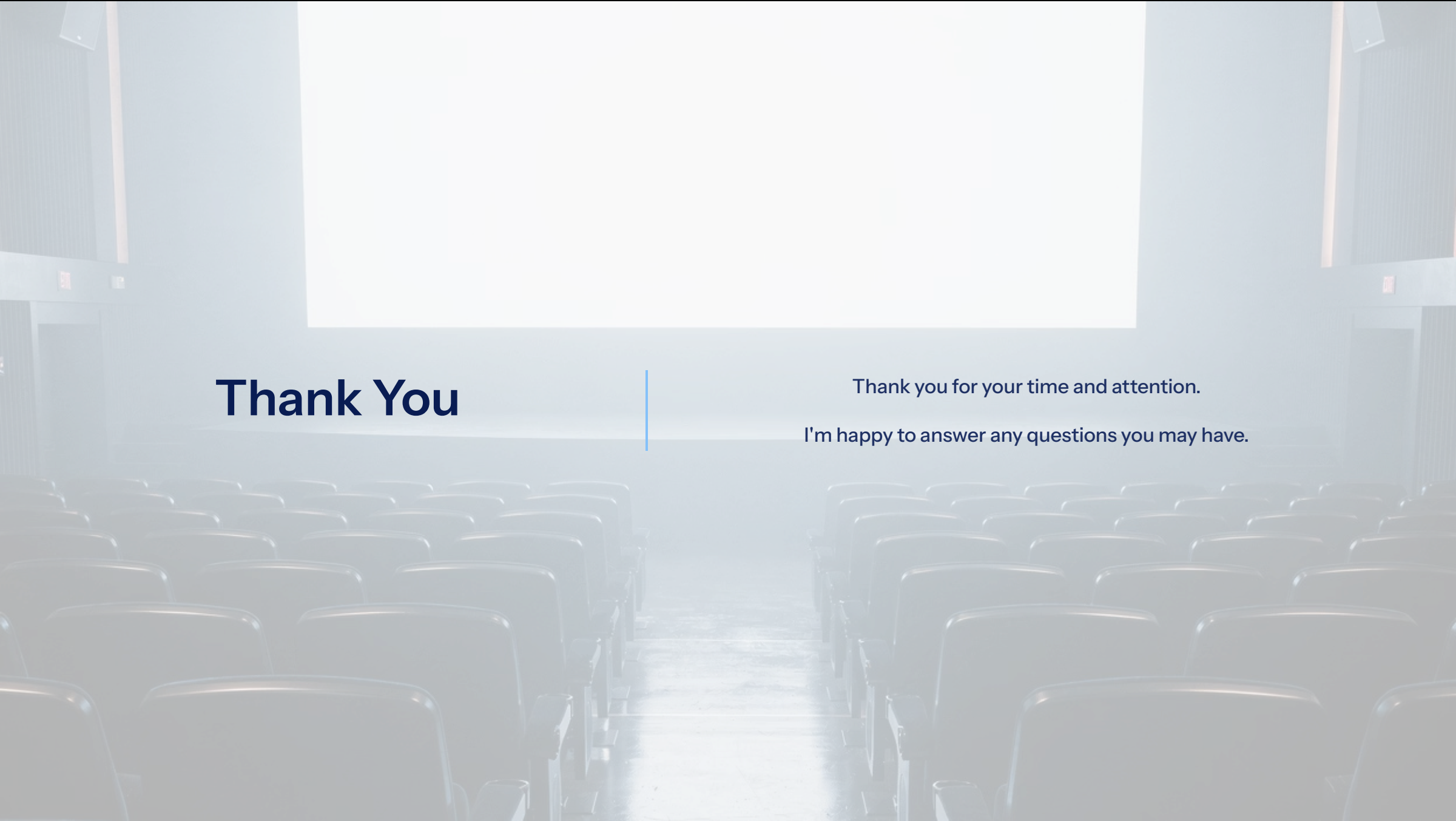


BERT / RoBERTa

Leverage transformer-based models for contextual understanding — tackling sarcasm and negation head-on.



Move toward contextual understanding to overcome the limitations of classical feature engineering.



Thank You

Thank you for your time and attention.
I'm happy to answer any questions you may have.